

Integrantes:

Santiago Eduardo Muñoz Castillo - 0000394453

Edward Augusto Ramirez Rodriguez - 0000324316

Wolfran Alirio Pinzon Murillo - 0000393439

Henry Julian Salazar Salcedo - 0000396117

Contenido

Contents

1.	Resumen ejecutivo.....	4
2.	Enfoque Arquitectónico	4
3.	Análisis de requisitos.....	5
3.1	Contexto de Ecommify.	5
3.2	Requisitos funcionales	5
3.2.1	Gestión de clientes:.....	5
3.2.2	Gestión de órdenes:	5
3.2.3	3.2.3 Gestión de pagos:	5
3.2.4	Gestión de productos:	6
3.2.5	Gestión de vendedores:.....	6
3.2.6	Gestión de reseñas:.....	6
3.3	Requisitos no funcionales.	6
3.3.1	Escalabilidad: La base de datos debe soportar grandes volúmenes de datos para pedidos, clientes, productos y pagos.	6
3.3.2	Integridad: La base de datos debe garantizar consistencia entre órdenes, pagos y productos, manejando claves primarias únicas, por ejemplo una orden no puede existir sin un cliente.....	6
3.3.3	Rendimiento: La base de datos debe responder eficientemente a consultas recurrentes realizadas como historial de ventas, historial de ordenes, productos más vendidos.....	6
3.3.4	Seguridad: La base de datos debe proteger información sensible relacionada a pagos, datos financieros, información de clientes y vendedores.	6
3.3.5	Disponibilidad: La base de datos debe permanecer disponible para las operaciones de compra y consulta.	6
3.3.6	Mantenibilidad: La base de datos debe permitir agregar nuevas entidades, modificar e integrar funcionalidades.....	6
3.4	Identificación de entidades del dominio.....	6
3.5	Tabla de volúmenes de datos esperados.....	8
4.	Diseño conceptual.	8
4.2	Diagrama ER completo de alta calidad.....	8
4.3	Descripción detallada de cada entidad y relación.....	9
4.4	Restricciones de negocio identificadas.	9
5.	Diseño lógico – Módulo PostgreSQL.	10

5.2	Esquema relacional normalizado (3FN).....	10
5.3	Aplicación de tipos avanzados.....	11
6.	Diseño lógico preliminar – Módulo MongoDB.	11
6.2	Colecciones principales identificadas.	11
6.3	Esquema de documentos.....	12
6.4	Patrones de modelado a aplicar.	13
6.5	Justificación de datos MongoDB vs PostgreSQL.....	14
7.	Decisiones arquitectónicas.....	14
7.2	Análisis del teorema CAP.	14
7.3	Trade-offs y estrategias de mitigación.	14
8.	Diccionario de datos.....	15
8.2	Script SQL preliminares.....	15

1. Resumen ejecutivo.

El conjunto de datos público del comercio electrónico brasileño por Olist constituye un recurso estratégico para el diseño de arquitecturas de datos híbridas que integren capacidades transaccionales y analíticas.

Nuestra decisión clave fue adoptar un enfoque políglota, donde las operaciones críticas de negocio (pedidos, pagos, logística) se gestionan en un sistema relacional con integridad referencial, mientras que los datos no estructurados (reseñas y texto libre) se almacenan en un motor NoSQL para maximizar flexibilidad y escalabilidad.

La arquitectura se organiza en capas: ingestión de datos crudos en un data lake, y persistencia dual en bases transaccionales y analíticas. Este diseño permite responder tanto a consultas operativas de alta concurrencia como a análisis exploratorios y predictivos.

La integración con un data warehouse facilita la construcción de dashboards de desempeño logístico y satisfacción del cliente, mientras que el uso de técnicas de procesamiento de lenguaje natural habilita la extracción de insights de reseñas.

La estrategia de alta disponibilidad se asegura mediante replicación y particionamiento, garantizando resiliencia frente a cargas variables. En resumen, las decisiones arquitectónicas nos priorizan modularidad, escalabilidad y capacidad de evolución, ofreciendo un marco sólido para investigación académica y aplicaciones empresariales en el ecosistema de e-commerce.

2. Enfoque Arquitectónico

El dataset de Olist se presta para un enfoque arquitectónico políglota y analítico-transaccional. Nuestra propuesta se centra en una arquitectura modular y desacoplada, donde cada dominio (clientes, pedidos, pagos, logística, reseñas) se modela como un bounded context independiente.

- **Capa de ingestión:** Los CSV se integran en un data lake (ej. S3, GCS) para almacenamiento bruto.
- **Procesamiento:** Se emplea ETL/ELT con herramientas como Apache Spark o dbt para limpieza y normalización.
- **Modelo transaccional:** PostgreSQL o MySQL para consultas operativas y consistencia referencial.
- **Alta disponibilidad:** Replicación y particionamiento para soportar cargas concurrentes.
- **Flexibilidad políglota:** SQL para integridad y NoSQL (MongoDB) para reseñas textuales y análisis de sentimiento (texto libre).

3. Análisis de requisitos.

3.1 Contexto de Ecommify.

Ecommify es una plataforma de comercio electrónico multivendedor, orientada a productos tecnológicos, que usamos con fines académicos en el curso de “Diseño y optimización de base de datos”.

El sistema debe soportar el ciclo completo de una venta real:

- Registro de Clientes
- Publicación de productos por vendedores y administración del catalogo
- Creación de pedidos con uno o múltiples items
- Procesamiento de pagos
- Registro de reseñas o calificaciones postventa

El tipo de datos en nuestro sistema es variado. Contamos con datos transaccionales como los pedidos y pagos, que están muy estructurados y se relacionan entre sí. Además, tenemos datos de análisis como el catálogo y las reseñas. Al manejar estos datos de tipos de información decidimos usar una arquitectura híbrida: PostgreSQL para lo transaccional porque necesita integridad y relaciones claras y MongoDB para los análisis porque permite consultas más flexibles.

Como no contamos con un entorno productivo propio, usaremos el dataset público “Brazilian E-Commerce” (Olist) de Kaggle con aproximadamente 99.000 pedidos realizados entre 2016 - 2018, alineado con el dominio de Ecommify para este ejercicio.

3.2 Requisitos funcionales

3.2.1 Gestión de clientes:

- Registrar clientes
- Consultar historial de compras

3.2.2 Gestión de órdenes:

- Crear órdenes
- Asociar productos a una orden
- Gestionar estados de las órdenes

3.2.3 3.2.3 Gestión de pagos:

- Registrar pagos de las órdenes
- Manejar tipos de pagos

3.2.4 Gestión de productos:

- Registrar productos
- Clasificar productos por categorías
- Consultar información de productos

3.2.5 Gestión de vendedores:

- Registrar vendedores
- Asociar vendedores a productos vendidos
- Consultar ventas por vendedor

3.2.6 Gestión de reseñas:

- Permitir calificaciones de pedidos
- Registrar comentarios de clientes
- Consultar calificaciones de productos

3.3 Requisitos no funcionales.

3.3.1 **Escalabilidad:** La base de datos debe soportar grandes volúmenes de datos para pedidos, clientes, productos y pagos.

3.3.2 **Integridad:** La base de datos debe garantizar consistencia entre órdenes, pagos y productos, manejando claves primarias únicas, por ejemplo una orden no puede existir sin un cliente.

3.3.3 **Rendimiento:** La base de datos debe responder eficientemente a consultas recurrentes realizadas como historial de ventas, historial de ordenes, productos más vendidos.

3.3.4 **Seguridad:** La base de datos debe proteger información sensible relacionada a pagos, datos financieros, información de clientes y vendedores.

3.3.5 **Disponibilidad:** La base de datos debe permanecer disponible para las operaciones de compra y consulta.

3.3.6 **Mantenibilidad:** La base de datos debe permitir agregar nuevas entidades, modificar e integrar funcionalidades.

3.4 Identificación de entidades del dominio.

- **customers**

Representa a cada cliente con identificador único (customer_id). Guarda datos de ubicación aproximada (código postal, ciudad y estado). Un cliente puede realizar varios pedidos. Las reglas de negocio exigen que cada cliente sea único en el sistema y que todo pedido tenga un cliente asociado.

- **orders**

Cada pedido tiene identidad propia (ID orden). Encapsula lógica de negocio: estados (pendiente, enviado, entregado), fechas, relación con pagos y productos.

- **payments**

Cada pago tiene identidad propia (ID pago). Reglas de negocio: métodos de pago, validaciones.

- **products**

Identidad única (ID producto). Lógica de negocio: stock, precio, categoría. Evoluciona en el tiempo pero sigue siendo el mismo producto.

- **sellers**

Cada vendedor tiene identidad propia (ID vendedor). Reglas de negocio, relación con órdenes.

- **order_items**

Representa cada línea de un pedido: Que producto se vendió, que vendedor lo envió, precio del envío. Un pedido puede tener uno o varios productos. Se identifica con clave compuesta order_id + order_item_id. Es la entidad que conecta pedidos, productos y vendedores

- **order_reviews**

Almacena la calificación y los comentarios del cliente después de la compra. Está ligada a un pedido (order_id). Incluye puntuación (review_score) y texto de entrada libre en título y mensaje. Sirve para medir satisfacción y análisis de opiniones.

- **product_category**

Clasifica los productos del catálogo. Permite agrupar artículos por tipo

- **geolocation**

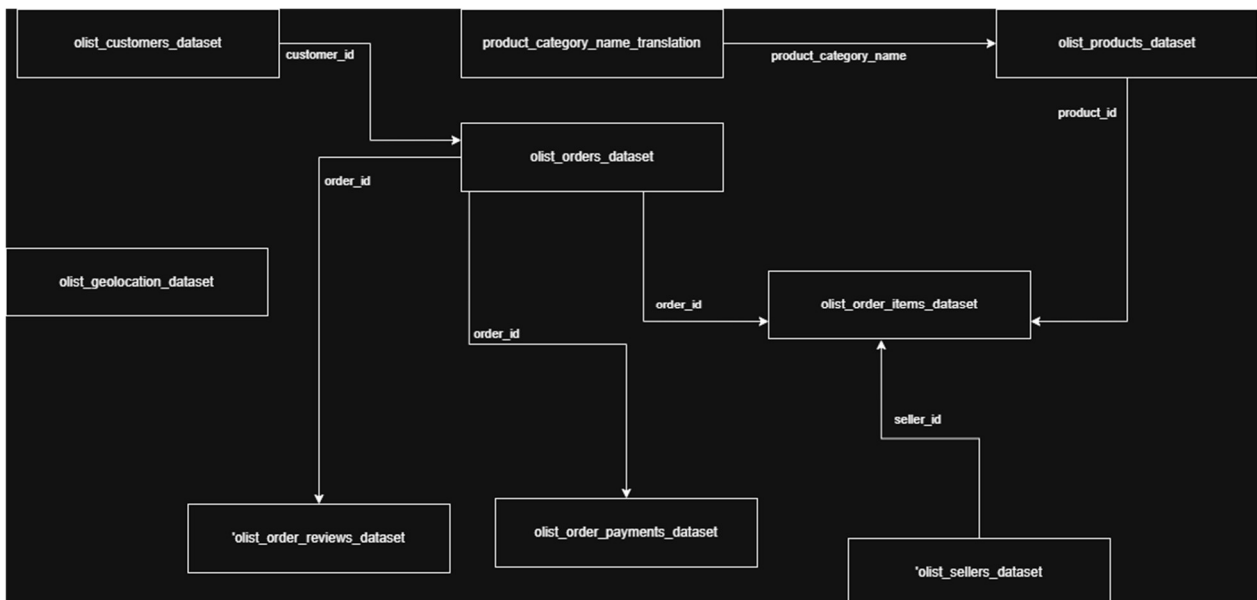
Contiene información de ubicación asociada a código postal. No se relaciona directamente uno a uno a cada pedido, se usa como referencia para análisis logístico y geográfico. En olist puede haber varios registros por el mismo prefijo postal.

3.5 Tabla de volúmenes de datos esperados.

<i>Entidad</i>	<i>Crecimiento diario</i>	<i>Crecimiento Anual aproximado</i>	<i>Tamaño promedio por registro</i>	<i>Volumen estimado anual</i>
customers	50	18250	2 KB	36.5 MB
orders	500	182500	3 KB	547.5 MB
payments	500	182500	2 KB	365 MB
products	10	3650	4 KB	16.7 MB
sellers	2	730	2 KB	1,46 MB

4. Diseño conceptual.

4.2 Diagrama ER completo de alta calidad.



4.3 Descripción detallada de cada entidad y relación.

- **Tabla customers**

Clave Primaria: customer_id

Atributos: customer_unique_id, customer_zip_code_prefix, customer_city, customer_state

- **Tabla orders**

Clave Primaria: order_id

Clave Foránea: customer_id

Atributos: order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date

- **Tabla order_items**

Clave Primaria Compuesta: order_id, order_item_id

Clave Foránea: product_id, seller_id

Atributos: shipping_limit_date, price, freight_value

- **Tabla order_payments**

Clave Primaria Compuesta: order_id, payment_sequential

Clave Foránea: order_id

Atributos: payment_type, payment_installments, payment_value

4.4 Restricciones de negocio identificadas.

Segregación de datos por tipo: Existe una clara decisión empresarial de separar los datos en «transaccionales» para PostgreSQL y «no transaccionales» para posibles soluciones NoSQL. Esto implica que los diferentes tipos de datos presentan distintos patrones de acceso, requisitos de flexibilidad y necesidades de rendimiento.

Entidades transaccionales (PostgreSQL): customers, orders, order_items y payments se consideran datos operativos fundamentales que requieren una alta integridad y consistencia relacional.

Entidades no transaccionales (candidatas a NoSQL): products, sellers, reviews y producto_categories se identifican como entidades con características (por ejemplo, atributos evolutivos, contenido de texto libre, jerarquías flexibles) más adecuadas para bases de datos NoSQL.

Integridad y unicidad de los datos: se aplican normas estrictas en cuanto a la integridad y la unicidad de los datos:

Claves primarias: todas las entidades identificadas (customers, orders, order_items payments, products, sellers) tienen claves primarias definidas (algunas compuestas) que deben ser únicas y no nulas. Esto garantiza que cada registro sea identificable de forma única.

Integridad referencial: se utilizan claves externas para vincular tablas relacionadas (por ejemplo, customer_id en orders se vincula a customers, order_id en order_items y payments se vincula a orders), lo que garantiza la coherencia en los datos transaccionales.

Restricciones de no nulo para claves: Las columnas de identificación críticas (especialmente aquellas que forman parte de claves primarias como order_item_id o payment_sequential) no pueden ser nulas, lo que conlleva la eliminación de filas si faltan estos valores.

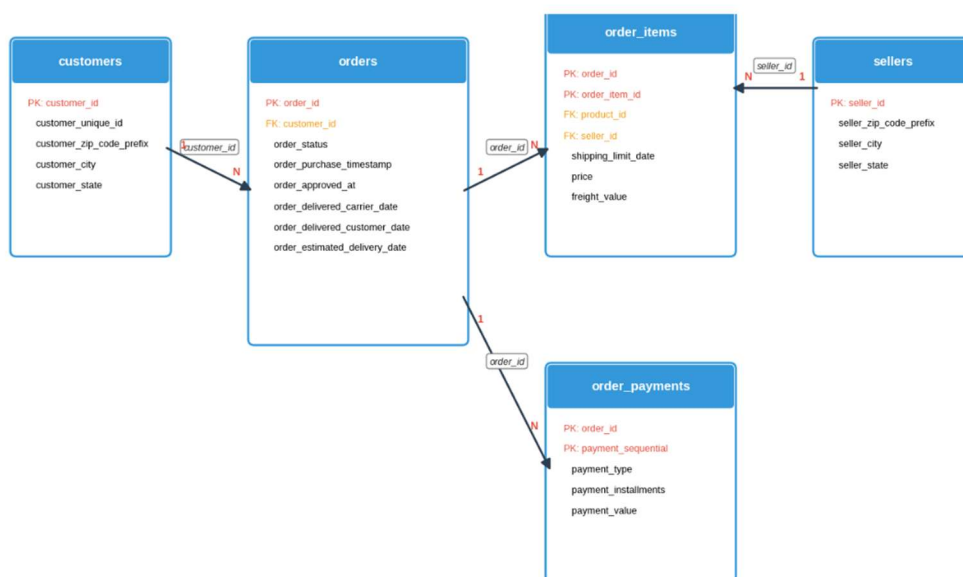
Datos atómicos (1FN): Las fechas y marcas de tiempo se convierten a tipos de fecha y hora adecuados para garantizar la atomicidad y permitir operaciones de tiempo correctas.

Gestión de duplicados: Las filas duplicadas se eliminan explícitamente basándose en identificadores clave (drop_duplicates() en customer_id, order_id, order_item_id, payment_sequential), lo que indica una regla de negocio contra los datos redundantes.

5. Diseño lógico – Módulo PostgreSQL.

5.2 Esquema relacional normalizado (3FN).

Diagrama Entidad Relación: Modelo Normalizado (3FN)



5.3 Aplicación de tipos avanzados.

Para el diseño de la base de datos relacional no se utilizarán tipos de datos especiales de PostgreSQL como por ejemplo JSONB o Arrays debido a que se adoptó una arquitectura con enfoque híbrido de persistencia de datos.

El uso de JSONB o Arrays dentro de PostgreSQL podría generar:

- Mayor complejidad en consultas y mantenimiento.
- Dificultades en la validación e integridad de datos.
- Menor claridad en el diseño arquitectónico de persistencia.

La información estructurada y transaccional del sistema será almacenada en PostgreSQL, aprovechando sus capacidades ACID, integridad referencial y consistencia transaccional. Este enfoque es adecuado para entidades altamente normalizadas como clientes, pedidos, pagos e inventario.

6. Diseño lógico preliminar – Módulo MongoDB.

6.2 Colecciones principales identificadas.

- products
Contiene el catálogo de productos. Sus atributos pueden ser complejos y variar, lo que se adapta bien a un esquema flexible de NoSQL.
- order_reviews
Contiene las valoraciones y comentarios de los clientes. El contenido de texto libre y la estructura flexible de los comentarios son ideales para NoSQL.
- product_categories
Contiene la jerarquía y traducción de categorías de productos. Puede ser mejor manejado en NoSQL si la estructura de categorías es dinámica o anidada.
- geolocation
Contiene información sobre los códigos postales brasileños y sus coordenadas de latitud y longitud. al ser manejado en NoSQL (MongoDB) puedes calcular las distancias y trazar mapas de distribución de ventas.

6.3 Esquema de documentos.

Colección products

```
{
  "product_id": "ABC123",
  "product_category": {
    "name": "beleza_saude",
    "name_english": "health_beauty"
  },
  "product_name_length": 25,
  "product_description_length": 120,
  "product_photos_qty": 3,
  "dimensions": {
    "weight_g": 1500,
    "length_cm": 30,
    "height_cm": 10,
    "width_cm": 20
  },
  "metadata": {
    "created_at": "2026-05-23T08:06:00Z",
    "updated_at": "2026-05-23T08:06:00Z"
  }
}
```

Se realiza agrupación de las medidas físicas en un subdocumento de “dimensions” con el fin de mantener ordenado el esquema y de fácil lectura. Así mismo, embebe product_category dado que es información pequeña y estable.

Campos Obligatorios: product_id, product_category_name, dimensions.

Validaciones: todos los valores numéricos deben ser enteros y mayores o iguales a 0.

Metadatos: fechas de creación y actualización se almacenan de tipo date para facilitar las consultas por rango de tiempo

Colección order_reviews

```
{
  "review_id": "R12345",
  "order_id": "O98765",
  "customer_id": "C54321",
  "review_score": 4,
  "review_comment_title": "Entrega rápida",
  "review_comment_message": "El producto llegó en buen estado.",
  "review_creation_date": "2026-05-20T10:30:00Z",
}
```

```
"review_answer_timestamp": "2026-05-21T09:15:00Z"
}
```

Campos Obligatorios: review_id, order_id, customer_id, review_score

Validaciones: las puntuaciones de score se realizan entre 1 a 5

Metadatos: fechas de creación y actualización se almacenan de tipo date para facilitar las consultas por rango de tiempo

Comentarios: Son opcionales, pero útiles para análisis de sentimientos y minería de datos

Colección geolocation

```
{
  "geolocation_zip_code_prefix": "01001",
  "location": {
    "type": "Point",
    "coordinates": [-46.6333, -23.5505]
  },
  "geolocation_city": "São Paulo",
  "geolocation_state": "SP",
  "metadata": {
    "created_at": "2026-05-23T08:12:00Z",
    "updated_at": "2026-05-23T08:12:00Z"
  }
}
```

Campos Obligatorios: geolocation_zip_code_prefix, location

location: Se guarda como un objeto GeoJSON (point) con coordenadas [longitud, latitud].

Metadatos: fechas de creación y actualización se almacenan de tipo date para facilitar las consultas por rango de tiempo

Este esquema se comporta como diccionario de referencia de productos.

6.4 Patrones de modelado a aplicar.

Colección	Patrón	Razón
products	Embedding	Products embebe a product_category, dado que siempre se muestra el producto y esto optimiza las lecturar al eliminar joins y garantiza atomicidad en las actualizaciones del documento completo.

order_reviews	Referencing	Se referencian identificadores que apuntan a orders y customers, dado que los datos almacenados son voluminosos (orders) y pueden actualizarse de forma independiente (customers)
geolocation	Computed	Almacena resultados precalculados evitando realizar operaciones posteriores

6.5 Justificación de datos MongoDB vs PostgreSQL.

Ecommify adopta una arquitectura híbrida porque no todos los datos se comportan igual, algunos requieren integridad y relaciones estrictas y otros se consulta de forma flexible. Para decidir el motor de persistencia usamos estos criterios:

- Integridad y transacciones: Si el dato es crítico para el negocio (pedidos,pagos) y debe cumplir ACID, va a PostgreSQL.
- Relaciones entre entidades: Si hay muchas claves foráneas y dependencias, va en PostgreSQL
- Flexibilidad de esquema: Si el contenido varía o conviene agrupar subdocumentos (dimensiones del producto, metadatos), va en MongoDB
- Patron de consulta: Si el uso principal es analítico o de lectura con opción de filtros variados, va en MongoDB



7. Decisiones arquitectónicas.

7.2 Análisis del teorema CAP.

Decisión de CAP por tipo de modulo:

- Modulo transaccional: prioridad Consistencia (C).
- Modulo analítico y documental: prioridad Disponibilidad (A).

7.3 Trade-offs y estrategias de mitigación.

Se acepta mayor complejidad de integración para obtener consistencia fuerte en operaciones críticas y flexibilidad en analítica.

8. Diccionario de datos.

8.2 Script SQL preliminares.

```
create table public.customers (  
  customer_id uuid not null,  
  customer_unique_id text null,  
  customer_zip_code_prefix integer null,  
  customer_city text null,  
  customer_state text null,  
  constraint customers_pkey primary key (customer_id)  
) TABLESPACE pg_default;
```

```
create table public.orders (  
  order_id uuid not null,  
  customer_id uuid null,  
  order_status text null,  
  order_purchase_timestamp timestamp without time zone null,  
  order_approved_at timestamp without time zone null,  
  order_delivered_carrier_date timestamp without time zone null,  
  order_delivered_customer_date timestamp without time zone null,  
  order_estimated_delivery_date timestamp without time zone null,  
  constraint orders_pkey primary key (order_id)  
) TABLESPACE pg_default;
```

```
create table public.order_payments (  
  order_id uuid not null,  
  payment_sequential numeric not null,  
  payment_type text null,  
  payment_installments numeric null,  
  payment_value numeric null,  
  constraint order_payments_pkey primary key (order_id, payment_sequential)  
) TABLESPACE pg_default;
```

```
create table public.order_items (  
  order_id uuid not null,  
  order_item_id integer not null,  
  product_id uuid null,  
  seller_id uuid null,  
  shipping_limit_date timestamp without time zone null,  
  price numeric null,  
  freight_value numeric null,  
  constraint order_items_pkey primary key (order_id, order_item_id)  
) TABLESPACE pg_default;
```

```
CREATE TABLE IF NOT EXISTS order_items_p0  
PARTITION OF order_items  
FOR VALUES WITH (MODULUS 4, REMAINDER 0);
```

```
CREATE TABLE IF NOT EXISTS order_items_p1  
PARTITION OF order_items  
FOR VALUES WITH (MODULUS 4, REMAINDER 1);
```

```
CREATE TABLE IF NOT EXISTS order_items_p2  
PARTITION OF order_items  
FOR VALUES WITH (MODULUS 4, REMAINDER 2);
```

```
CREATE TABLE IF NOT EXISTS order_items_p3
```

```
PARTITION OF order_items  
FOR VALUES WITH (MODULUS 4, REMAINDER 3);
```

```
create table public.sellers (  
  seller_id uuid not null,  
  seller_zip_code_prefix numeric null,  
  seller_city text null,  
  seller_state text null,  
  constraint sellers_pkey primary key (seller_id)  
) TABLESPACE pg_default;
```